

# TP12 — Script de diagnostic réseau automatisé

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

## Description

Ce TP étend le client Python modulaire (TP3) avec un module de diagnostic automatisé : il interroge chaque site, mesure la latence, détecte les pannes (site injoignable, authentification refusée, interface en panne), et propose une action suggérée pour chaque type de problème.

## Prérequis

- TP1 à TP11 validés (le package `scripts/netops_client` du TP3 doit être présent — mergé depuis dev si besoin)

## Étapes

### Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour3-tp12
```

### Étape 2 — Séparer logs détaillés (fichier) et sortie console (erreurs seulement)

Pour que le tableau récapitulatif reste lisible, les logs INFO/WARNING de chaque requête n'iront désormais que dans le fichier de log, pas sur la console :

```
cat > scripts/netops_client/logger_config.py << 'EOF'
import logging
```

```
def setup_logger(name, log_file="scripts/logs_network_client.log"):
    """Cree et configure un logger reutilisable par tous les modules du package."""
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger
```

```

logger.setLevel(logging.INFO)
formatter = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s")

file_handler = logging.FileHandler(log_file)
file_handler.setFormatter(formatter)
file_handler.setLevel(logging.INFO)
logger.addHandler(file_handler)

# La console ne recoit que les erreurs, pas le bruit INFO/WARNING habituel
console_handler = logging.StreamHandler()
console_handler.setFormatter(formatter)
console_handler.setLevel(logging.ERROR)
logger.addHandler(console_handler)

return logger
EOF

```

### Étape 3 — Mettre à jour le client pour supporter l'authentification

```

cat > scripts/netops_client/client.py << 'EOF'
import time

import requests

from .exceptions import NetworkAPIError
from .logger_config import setup_logger

logger = setup_logger(__name__)

class NetworkClient:
    """Client modulaire pour interroger une API reseau REST."""

    def __init__(self, base_url, api_token=None, timeout=5):
        self.base_url = base_url.rstrip("/")
        self.api_token = api_token
        self.timeout = timeout

    def _headers(self):
        if self.api_token:
            return {"X-API-Token": self.api_token}
        return {}

    def get_health(self):
        url = f"{self.base_url}/health"
        start = time.monotonic()
        try:
            response = requests.get(url, timeout=self.timeout)

```

```

        elapsed_ms = round((time.monotonic() - start) * 1000, 1)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        logger.error(f"Echec du health check : {e}")
        raise NetworkAPIError(f"Site injoignable : {e}")

    return {"data": response.json(), "elapsed_ms": elapsed_ms}

def get_interfaces(self, device_id):
    url = f"{self.base_url}/api/devices/{device_id}/interfaces"
    logger.info(f"Requete GET vers {url}")
    try:
        response = requests.get(url, headers=self._headers(), timeout=self.timeout)
        response.raise_for_status()
    except requests.exceptions.HTTPError as e:
        status = e.response.status_code if e.response is not None else None
        if status == 401:
            logger.error(f"Authentification refusee sur {url}")
            raise NetworkAPIError("Authentification refusee (token invalide ou manquant)")
        logger.error(f"Erreur HTTP {status} sur {url}")
        raise NetworkAPIError(f"Erreur HTTP {status} : {e}")
    except requests.exceptions.RequestException as e:
        logger.error(f"Echec de la requete API : {e}")
        raise NetworkAPIError(f"Impossible de contacter l'API : {e}")

    data = response.json()
    logger.info(f"Reponse recue pour {device_id} : {len(data['interfaces'])} interfaces")
    return data["interfaces"]

def count_interfaces_down(self, device_id):
    interfaces = self.get_interfaces(device_id)
    down = [name for name, info in interfaces.items() if info["status"] == "down"]
    if down:
        logger.warning(f"Interfaces down sur {device_id} : {down}")
    return down

```

EOF

## Étape 4 — Créer le module de diagnostic avec règles de suggestion

```

cat > scripts/netops_client/diagnostics.py << 'EOF'
import datetime

from .client import NetworkClient
from .exceptions import NetworkAPIError
from .logger_config import setup_logger

logger = setup_logger(__name__)

```

```

def diagnose_site(site_name, base_url, api_token, device_id):
    """Execute un diagnostic complet d'un site et retourne un rapport structure."""
    report = {
        "site": site_name,
        "timestamp": datetime.datetime.now().isoformat(timespec="seconds"),
        "reachable": False,
        "latency_ms": None,
        "auth_ok": None,
        "interfaces_down": [],
        "suggested_actions": [],
    }

    client = NetworkClient(base_url=base_url, api_token=api_token)

    # Etape 1 : le site est-il joignable ?
    try:
        health = client.get_health()
        report["reachable"] = True
        report["latency_ms"] = health["elapsed_ms"]
        logger.info(f"{site_name} : joignable ({health['elapsed_ms']} ms)")
    except NetworkAPIError as e:
        report["suggested_actions"].append(
            "Site injoignable : verifier que le conteneur/service tourne, verifier le reseau/port"
        )
        logger.error(f"{site_name} : injoignable ({e})")
        return report

    if report["latency_ms"] and report["latency_ms"] > 200:
        report["suggested_actions"].append(
            f"Latence elevee ({report['latency_ms']} ms) : verifier la charge du site ou le reseau"
        )

    # Etape 2 : l'authentification et les interfaces
    try:
        down = client.count_interfaces_down(device_id)
        report["auth_ok"] = True
        report["interfaces_down"] = down
        if down:
            report["suggested_actions"].append(
                f"Interface(s) en panne : {down} -> verifier le cablage ou la configuration"
            )
    except NetworkAPIError as e:
        report["auth_ok"] = False
        if "Authentication" in str(e):
            report["suggested_actions"].append(
                "Authentication refusee : verifier que le token Ansible Vault correspond "
                "a celui configure sur le site (variable API_TOKEN)"
            )

```

```

        )
    else:
        report["suggested_actions"].append(f"Erreur lors de la lecture des interfaces : {e}")
        logger.error(f"{site_name} : {e}")

    if not report["suggested_actions"]:
        report["suggested_actions"].append("Aucune anomalie detectee")

    return report
EOF

```

## Étape 5 — Exposer le nouveau module dans le package

```

cat > scripts/netops_client/__init__.py << 'EOF'
from .client import NetworkClient
from .exceptions import NetworkAPIError
from .diagnostics import diagnose_site

__all__ = ["NetworkClient", "NetworkAPIError", "diagnose_site"]
EOF

```

## Étape 6 — Créer le script principal de diagnostic

```

pip install pyyaml

cat > scripts/run_diagnostics.py << 'EOF'
import json
import os
import sys

import yaml

from netops_client import diagnose_site

HERE = os.path.dirname(os.path.abspath(__file__))
SITES_FILE = os.path.join(HERE, "..", "inventory", "sites.yml")
DIAG_DIR = os.path.join(HERE, "..", "diagnostics")

API_TOKEN = os.environ.get("VAULT_API_TOKEN")
if not API_TOKEN:
    print("ERREUR : variable d'environnement VAULT_API_TOKEN non definie.")
    print("Exemple : export VAULT_API_TOKEN=s3cr3t-token-nexacorp")
    sys.exit(1)

def main():
    with open(SITES_FILE) as f:
        sites = yaml.safe_load(f)["sites"]

```

```

os.makedirs(DIAG_DIR, exist_ok=True)
reports = []

print(f"{'SITE':10s} {'JOIGNABLE':10s} {'LATENCE':10s} {'AUTH':6s} {'INTERFACES DOWN'}")
print("-" * 70)

for site in sites:
    base_url = f"http://localhost:{site['api_port']}"
    device_id = f"router-{site['nom']}-01"
    report = diagnose_site(site["nom"], base_url, API_TOKEN, device_id)
    reports.append(report)

    joignable = "oui" if report["reachable"] else "NON"
    latence = f"{report['latency_ms']} ms" if report["latency_ms"] else "-"
    auth = "ok" if report["auth_ok"] else ("NON" if report["auth_ok"] is False else "-")
    down = ", ".join(report["interfaces_down"]) if report["interfaces_down"] else "-"

    print(f"{site['nom']:10s} {joignable:10s} {latence:10s} {auth:6s} {down}")

print()
for report in reports:
    print(f"[{report['site']}] Actions suggerees :")
    for action in report["suggested_actions"]:
        print(f"  - {action}")

timestamp = reports[0]["timestamp"].replace(":", "-")
report_file = os.path.join(DIAG_DIR, f"diagnostic_{timestamp}.json")
with open(report_file, "w") as f:
    json.dump(reports, f, indent=2, ensure_ascii=False)
print(f"\nRapport complet ecrit dans {report_file}")

if any(not r["reachable"] or r["auth_ok"] is False or r["interfaces_down"] for r in reports):
    sys.exit(1)

if __name__ == "__main__":
    main()
EOF

```

## Étape 7 — Ignorer le dossier de rapports

```
echo "diagnostics/" >> .gitignore
```

## Étape 8 — Tester en conditions normales

```
cd lab && docker compose up -d --build && cd ..
export VAULT_API_TOKEN=s3cr3t-token-nexacorp
```

```
python scripts/run_diagnostics.py
```

**Résultat attendu :** un tableau récapitulatif propre (site / joignable / latence / auth / interfaces down), une liste d'actions suggérées par site, et un fichier JSON écrit dans diagnostics/.

Pour lire le rapport le plus récent (il y en a un nouveau à chaque exécution) :

```
cat "$(ls -t diagnostics/diagnostic_*.json | head -1)" | python -m json.tool
cat diagnostics/diagnostic_*.json seul concatènerait plusieurs rap-
ports JSON à la suite, ce qui n'est plus un JSON valide (Extra data).
```

## Étape 9 — Simuler un site injoignable

```
cd lab
docker compose stop agence2-router
cd ..
python scripts/run_diagnostics.py
```

**Résultat attendu :** agence2 : NON (injoignable, latence et auth à -), avec la suggestion Site injoignable : verifier que le conteneur/service tourne....

```
cd lab && docker compose start agence2-router && cd ..
```

## Étape 10 — Simuler un token invalide

### 10.1 — Vérifier l'état actuel

```
python scripts/run_diagnostics.py
```

La ligne siege doit afficher AUTH : ok.

### 10.2 — Casser volontairement le token de siege-router uniquement

```
sed -i '/siege-router:\/,\/networks:\/ s/API_TOKEN=\${API_TOKEN}/API_TOKEN=TOKEN_CASSE/' lab/docker-compose.yml
grep -A 3 "container_name" lab/docker-compose.yml
```

Vérifiez que seul siege-router a API\_TOKEN=TOKEN\_CASSE, les deux autres gardent API\_TOKEN=\${API\_TOKEN}.

### 10.3 — Recréer uniquement le conteneur siege-router

```
cd lab
docker compose up -d --build siege-router
cd ..
```

### 10.4 — Vérifier que la panne est en place

```
curl -s -H "X-Api-Token: s3cr3t-token-nexacorp" http://localhost:5001/api/devices/router-siege-01/interfaces
```

Doit répondre {"error": "Token invalide ou manquant"}.

## 10.5 — Lancer le diagnostic

```
python scripts/run_diagnostics.py
```

**Résultat attendu :** siege affiche oui (joignable, une latence s’affiche bien) mais AUTH : NON — distinct d’un site complètement injoignable où latence et auth affichent - tous les deux. La suggestion associée : Authentification refusée : vérifier que le token Ansible Vault correspond à celui configuré sur le site....

## 10.6 — Réparer et revenir à l’état normal

```
sed -i 's/API_TOKEN=TOKEN_CASSE/API_TOKEN=${API_TOKEN}/' lab/docker-compose.yml
cd lab
docker compose up -d --build siege-router
cd ..
python scripts/run_diagnostics.py
```

La ligne siege doit repasser à AUTH : ok.

## Étape 11 — Committer et pousser

```
git add scripts/ .gitignore
git commit -m "TP12 : script de diagnostic réseau automatisé avec suggestions"
git push -u origin <prenom>-<nom>-jour3-tp12
```

Ouvrir une pull request <prenom>-<nom>-jour3-tp12 → dev.

## Comprendre le TP

Le module `diagnostics.py` encapsule une logique de décision en deux temps, imitant le raisonnement d’un ingénieur qui investigate un incident : d’abord “le site répond-il ?” (si non, inutile d’aller plus loin) ; ensuite, seulement si le site répond, “l’authentification passe-t-elle, et les interfaces sont-elles dans l’état attendu ?”. Chaque type d’échec génère une suggestion différente et actionnable, plutôt qu’un message d’erreur générique — c’est ce qui distingue un outil de “debug automatisé” d’un simple test qui dit juste “échec”.

La séparation entre logs détaillés (fichier, tout niveau) et sortie console (erreurs uniquement) reprend un principe classique d’observabilité : la console sert à l’opérateur qui veut une réponse rapide et lisible, le fichier de log sert à l’investigation approfondie quand quelque chose ne va pas — les deux publics n’ont pas besoin du même niveau de détail au même moment.

Le rapport JSON écrit dans `diagnostics/` sert de trace historique : contrairement à l’affichage console qui disparaît, ce fichier peut être archivé, comparé d’un jour sur l’autre, ou intégré plus tard à un système de supervision.



## Validation

- ☐ `python scripts/run_diagnostics.py` affiche un tableau récapitulatif propre des 3 sites, sans les logs détaillés mélangés dedans
- ☐ Chaque site reçoit au moins une action suggérée (même "Aucune anomalie détectée" si tout va bien)
- ☐ Un site arrêté est détecté comme injoignable, avec la bonne suggestion
- ☐ Un token invalide est détecté comme échec d'authentification (distinct d'un site injoignable), avec la bonne suggestion
- ☐ Un fichier JSON de rapport est créé dans `diagnostics/` à chaque exécution
- ☐ Une pull request `<prenom>-<nom>-jour3-tp12` → dev est ouverte